A Hands-on Exercise: Using OpenMP for Programmer Directed Shared Memory Programming

Parallel and Distributed Computing (CS-3006)

OpenMP as "programmable automation" for shared-memory parallelism

- **What you already know:** pthreads, threads, joins, races, basic speedup concepts
- **What you will learn in this exercise:** how OpenMP controls (and hides) thread management, data scoping, synchronization cost, scheduling, and performance pitfalls

Learning Outcomes

By the end of this lab, you should be able to:

- Explain OpenMP's fork-join execution model and how it differs from pthreads.
- Predict and validate the number of threads created using precedence rules.
- Correctly use OpenMP data environment clauses: `shared`, `private`, `firstprivate`, `lastprivate`, `default(none)`, `reduction`.
- Fix race conditions using `critical`, `atomic`, and `reduction`, and compare their performance.
- Choose an appropriate scheduling strategy (`static`, `dynamic`, `guided`, `runtime`) for load balance vs overhead.
- Use `single`, `master`, and `barrier` constructs appropriately.
- Use `collapse` to parallelize nested loops.
- Measure speedup and efficiency and diagnose bottlenecks (sync, load imbalance, false sharing, memory bandwidth).

Setup and Compilation

Compiler (GCC)

Compile with OpenMP support:

```
gcc -O2 -fopenmp file.c -o prog
```

Thread Count Control

Set threads via environment:

```
export OMP_NUM_THREADS=4
./prog
```

Optional (useful for debugging / stable demo runs):

```
export OMP_PROC_BIND=true
export OMP_PLACES=cores
```

Timing Hint

Use `omp_get_wtime()` for timing.

```
double t0 = omp_get_wtime();
/* work */
double t1 = omp_get_wtime();
printf("Time: %.6f s\n", t1 - t0);
```

1 Part-A: Programmer Directed Parallelism From Parallel Thinking to OpenMP

OpenMP does not remove the need for parallel thinking. It *automates* part of the mechanics.

Quick Mapping: Relate each step to OpenMP:

1. **Problem Understanding:** Obviously, our own task!
2. **Decomposition:** _____
3. **Assignment:** _____
4. **Orchestration:** _____
5. **Mapping:** _____

Deliverable

A short paragraph: "What OpenMP automates vs what remains my job."

2 Part B: Fork-Join Thread Teams and Thread Count Precedence

2.1 How Many Threads?

The number of threads in a parallel region is determined by the following factors, **in order of precedence**:

1. Evaluation of the `if` clause
2. Setting of the `num_threads` clause
3. Use of the `omp_set_num_threads()` library function
4. Setting of the `OMP_NUM_THREADS` environment variable
5. Implementation default (usually number of CPUs on the node)

Threads are numbered from **0** (master thread) to **N-1**.

2.2 Task B1: Validate the Precedence Rules

Save as `B1_threads.c`.

```c
#include <stdio.h>
#include <omp.h>

int main() {
    omp_set_num_threads(2);

    // Change if(1) to if(0) in one run:
    #pragma omp parallel if(1) num_threads(4)
    {
        #pragma omp critical
        printf("Hello from T=%d (team size=%d)\n",
               omp_get_thread_num(), omp_get_num_threads());
    }
    return 0;
}
```

Run Plan (record results):

1. Run with: `export OMP_NUM_THREADS=8`
2. Compile and execute.
3. Change `if(1)` to `if(0)` and re-run.

Questions:

1. Which mechanism determined the final thread count in each run?
2. What happened when `if(0)` was used, and why?
3. Why is thread 0 called the “master” thread?

2.3 Task B2 (Micro): `single` vs `master`

Add the following inside the parallel region of Task B1 (below the printf), then re-run:

```
#pragma omp single
printf("This prints once (single). Executed by: %d\n",
    omp_get_thread_num());

#pragma omp master
printf("This prints once (master). Executed by: %d\n",
    omp_get_thread_num());
```

Question: What is the difference between `single` and `master` regarding barriers?

3 Part C: Data Environment Clauses (Shared vs Private vs Firstprivate vs Lastprivate)

3.1 Data Environment Clauses

OpenMP provides clauses to control data visibility:

- `private(list of variables)`
- `firstprivate(list of variables)`
- `lastprivate(list of variables)`
- `shared(list of variables)`
- `default(none | shared | private)`
- `copyin(list of variables)`
- `reduction(operator : list)`

3.2 Task C1: Observe `private` vs `shared`

Save as `C1_scope.c`.

```c
#include <stdio.h>
#include <omp.h>

int main() {
    int x = 10;

    #pragma omp parallel private(x)
    {
        x = 100 + omp_get_thread_num();
    }

    printf("After parallel region, x = %d\n", x);
    return 0;
}
```

Now modify and re-run:

1. Remove `private(x)` (use default sharing).
2. Replace with `firstprivate(x)`.

Questions:

1. Why does `private(x)` not update the original `x`?
2. Why can removing `private(x)` produce weird/non-deterministic outcomes?
3. What exactly does `firstprivate(x)` guarantee?

3.3 Task C2: Enforce Good Practice with `default(none)`

Rewrite your parallel region using `default(none)` and explicitly declare each variable:

```c
#pragma omp parallel default(none) shared(...) private(...)
```

Question: Why is `default(none)` considered "professional-mode OpenMP"?

3.4 Task C3 (Micro): Demonstrate `lastprivate`

Create a loop where each iteration writes to a variable, and use `lastprivate` to capture the value from the **last** iteration (logical last, not "last thread"):

```
int last = -1;
#pragma omp parallel for lastprivate(last)
for(int i=0;i<10;i++){
    last = i;
}
printf("last = %d\n", last); // expected 9
```

Question: Why is `lastprivate` different from "whatever finishes last"?

4 Part D: Race Conditions and Fixes (Correctness & Cost)

4.1 Task D1: The Classic Race

Save as `D1_race.c`.

```c
#include <stdio.h>
#include <omp.h>

int main() {
    int sum = 0;

    #pragma omp parallel for
    for (int i = 0; i < 1000000; i++)
        sum++;

    printf("Sum = %d\n", sum);
    return 0;
}
```

Run multiple times. It may vary.

4.2 Task D2: Fix Using `critical`

Save as `D2_critical.c`. Use timing.

```c
#include <stdio.h>
#include <omp.h>

int main() {
    int sum = 0;
    double t0 = omp_get_wtime();

    #pragma omp parallel for
    for (int i = 0; i < 1000000; i++) {
        #pragma omp critical
        sum++;
    }

    double t1 = omp_get_wtime();
    printf("Sum=%d Time=%.6f\n", sum, t1-t0);
    return 0;
}
```

4.3 Task D3: Fix Using `atomic`

Save as `D3_atomic.c`.

```c
#include <stdio.h>
#include <omp.h>

int main() {
    int sum = 0;
    double t0 = omp_get_wtime();

    #pragma omp parallel for
    for (int i = 0; i < 1000000; i++) {
        #pragma omp atomic
        sum++;
```

```
    }

    double t1 = omp_get_wtime();
    printf("Sum=%d Time=%.6f\n", sum, t1-t0);
    return 0;
}
```

4.4 Task D4: Fix Using `reduction`

Save as `D4_reduction.c`.

```
#include <stdio.h>
#include <omp.h>

int main() {
    int sum = 0;
    double t0 = omp_get_wtime();

    #pragma omp parallel for reduction(+:sum)
    for (int i = 0; i < 1000000; i++)
        sum++;

    double t1 = omp_get_wtime();
    printf("Sum=%d Time=%.6f\n", sum, t1-t0);
    return 0;
}
```

Analysis Table (Fill)

Method	Correct?	Time (s)	Notes (Why fast/slow?)
No sync			
critical			
atomic			
reduction			

Core Questions:

1. Why does `critical` usually scale poorly?
2. Why is `reduction` often the best for sums?
3. In what cases is `atomic` not enough?

4.5 Task D5 (Micro): Reduction on `max`

Implement a `max` reduction on a random array (or a fixed array). Use:

```
#pragma omp parallel for reduction(max:mx)
```

Question: Why does `reduction(max:mx)` avoid locking?

5 Part E: Scheduling and Load Balancing

5.1 Task E1: Imbalanced Loop (Static vs Dynamic vs Guided)

Save as `E1_sched.c`.

```c
#include <stdio.h>
#include <omp.h>

static inline void heavy_work(int i) {
    volatile long sink = 0;
    long iters = (long)i * 2000L; // imbalance increases with i
    for (long j = 0; j < iters; j++) sink += j;
}

int main() {
    const int N = 20000;

    double t0 = omp_get_wtime();

    // Try different schedules by editing the directive:
    #pragma omp parallel for schedule(static)
    for (int i = 0; i < N; i++) {
        heavy_work(i);
    }

    double t1 = omp_get_wtime();
    printf("Time = %.6f\n", t1 - t0);
    return 0;
}
```

Experiments (record times):

- `schedule(static)`
- `schedule(static, 50)`
- `schedule(dynamic, 50)`
- `schedule(guided)`

Schedule	Time (s)
static	
static,50	
dynamic,50	
guided	

Questions:

1. Which scheduling strategy performed best and why?
2. When can `dynamic` become slower than `static`?
3. What is the tradeoff between **load balance** and **scheduling overhead**?

5.2 Task E2 (Micro): `schedule(runtime)` and `OMP_SCHEDULE`

Change directive to:

```
#pragma omp parallel for schedule(runtime)
```

Then test without recompiling:

```
export OMP_SCHEDULE="static,50"
export OMP_SCHEDULE="dynamic,50"
export OMP_SCHEDULE="guided"
```

Question: Why is `schedule(runtime)` useful for performance tuning?

6 Part F: Implicit Barrier and `nowait`

Most OpenMP work-sharing constructs end with an **implicit barrier**. Sometimes that barrier is unnecessary.

6.1 Task F1: Compare With/Without Barrier

Save as `F1_nowait.c`.

```c
#include <stdio.h>
#include <omp.h>

int main() {
    const int N = 20000000;
    static double A[20000000];
    static double B[20000000];

    double t0 = omp_get_wtime();

    #pragma omp parallel
    {
        #pragma omp for
        for (int i = 0; i < N; i++) A[i] = i * 0.5;

        // Try: add "nowait" above and see if time changes in your
           environment
        #pragma omp for
        for (int i = 0; i < N; i++) B[i] = A[i] + 1.0;
    }

    double t1 = omp_get_wtime();
    printf("Time = %.6f\n", t1 - t0);
    return 0;
}
```

Questions:

1. Why is there a barrier after the first `omp for`?
2. When is `nowait` safe?
3. When can removing a barrier break correctness?

6.2 Task F2 (Micro): Explicit `barrier`

Insert an explicit barrier and observe the impact (timing + reasoning):

```c
#pragma omp barrier
```

Question: When is an explicit barrier necessary, even if it hurts performance?

7 Part G: False Sharing (Performance Pitfall)

False sharing happens when threads update different variables that reside on the **same cache line**, causing cache-coherence ping-pong.

7.1 Task G1: Without Padding

Save as `G1_false_sharing.c`.

```c
#include <stdio.h>
#include <omp.h>

int main() {
    const long N = 200000000L;
    static double partial[64]; // assumes <=64 threads; adjust if needed

    double t0 = omp_get_wtime();

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        for (long i = 0; i < N; i++) {
            partial[tid] += 1.0;
        }
    }

    double t1 = omp_get_wtime();
    printf("Time (no padding) = %.6f\n", t1 - t0);
    return 0;
}
```

7.2 Task G2: With Padding

Modify to:

```c
static double partial[64][8];
partial[tid][0] += 1.0;
```

Questions:

1. Why can padding speed this up dramatically?
2. What is a cache line and why does it matter here?

8 Part H: Nested Loops with `collapse`

8.1 Task H1: Parallelize a 2D Kernel

Consider a 2D grid update (e.g., image-like / matrix-like):

```
for (int i = 0; i < R; i++)
  for (int j = 0; j < C; j++)
    B[i*C + j] = A[i*C + j] + 1.0;
```

Parallelize using:

```
#pragma omp parallel for collapse(2)
```

Questions:

1. What does `collapse(2)` change in the iteration space?
2. When might `collapse` improve load balance?
3. When might `collapse` increase overhead?

9 Part I: Matrix-Vector Multiplication (OpenMP Speedup)

9.1 Task I1: Implement Serial MatVec

Compute $y = Ax$ where A is $N \times N$.
Constraints:

- Use dynamic allocation for A, x, y
- Initialize with deterministic values
- Time only the matvec (not allocation/initialization)

9.2 Task I2: Parallelize with OpenMP

Parallelize the outer loop:

```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    double sum = 0.0;
    for (int j = 0; j < N; j++) sum += A[i*N + j] * x[j];
    y[i] = sum;
}
```

Report Metrics

Compute:

$$Speedup = \frac{T_{serial}}{T_{parallel}} \qquad Efficiency = \frac{Speedup}{P}$$

Threads (P)	$T_{parallel}$ (s)	Speedup	Efficiency
1			
2			
4			
8			

Questions:

1. Does speedup scale linearly? Why/why not?
2. Is this kernel more compute-bound or memory-bound on your machine?
3. Which earlier pitfall (sync/load imbalance/false sharing/bandwidth) is most relevant here?

Self-Reading: OpenMP SIMD Capabilities (Vectorization Pragmas)

Why SIMD with OpenMP?

OpenMP is not only about *threads*. It also provides directives to help the compiler generate *SIMD* (Single Instruction, Multiple Data) vector instructions (e.g., SSE/AVX/AVX2/AVX-512). This is **instruction-level parallelism** inside a single core, while `omp parallel` is **thread-level parallelism** across cores.

Key SIMD Constructs

- `#pragma omp simd`: request SIMD vectorization of the following loop.
- `#pragma omp parallel for simd`: combine multi-threading + SIMD in one loop.
- `#pragma omp declare simd`: create SIMD variants of a function (vector-callable).

Core Pragmas and Common Clauses

1) `#pragma omp simd` Use when loop iterations are independent (no loop-carried dependence).

```
#pragma omp simd
for (int i = 0; i < N; i++) {
    C[i] = A[i] * B[i] + C[i];
}
```

Useful clauses:

- `reduction(op: varlist)`: vector-friendly reduction.
- `private(varlist)` / `lastprivate(varlist)`
- `linear(varlist[:step])`
- `aligned(ptrlist[:alignment])`
- `simdlen(L)` / `safelen(L)`
- `collapse(k)`

```
#pragma omp parallel for simd
for (int i = 0; i < N; i++) {
    Y[i] = alpha * X[i] + Y[i];
}
```

```
#pragma omp declare simd
double f(double x) { return x*x + 1.0; }

#pragma omp simd
for (int i = 0; i < N; i++) {
    out[i] = f(in[i]);
}
```

How to Check if Vectorization Happened (Mini Task)

Compile your SIMD test file with vectorization reports:

```
gcc -O3 -fopenmp -fopt-info-vec-optimized -fopt-info-vec-missed code.c
    -o code
```

Question: What did the compiler vectorize, and what did it refuse to vectorize (and why)?